FRED TALKS

6th September 2026

CONTENTS

What does "playing politics" mean for software engineers?

SEANGOEDECKE.COM RSS FEED · 14 JUL 2026 · SOURCE

Software engineers are often told to “start playing politics”, but most engineers have no idea what that means.

Their reference point for “playing politics” comes from fiction like Game of Thrones. Are they supposed to raise an army and depose the CEO, or poison each other at team lunch? Should they book Zoom calls with each other and plot schemes? All of that is obviously ridiculous. In terms of Game of Thrones, software engineers are not lords and ladies. We’re the soldiers and workers of the realm. So you should think about “playing politics” in the way a castle guard would, not one of the major players.

The castle guard are not going around poisoning people or forming coalitions between the great powers. They are largely keeping their heads down. But in order to do that, they have to stay aware of the political currents, or they’re liable to do something catastrophically stupid: for instance, making an enemy of a powerful courtier, or arresting somebody who’s on an important mission for the king.

Given that, the basic principles of playing politics are something like this:

- Be aware of who’s powerful and who’s not
- At all costs, avoid making powerful enemies
- Help powerful people as best you can
- Make sure they know you’re helping them (without annoying them)

Be aware of who’s powerful and who’s not

As a software engineer in a large company, **you will not be a powerful person**. Powerful people are typically in senior management: VPs, directors, and so on[1]. However, not everyone in senior management is powerful. Some are killers who have the active support of the CEO, while others are confused incompetents.

That kicked off a number of days of iteration. Claude would propose a change to Ada. Ada would review Claude's spec, raising questions or concerns. Claude would update the spec. Once Claude and Ada were in agreement, Claude would build an updated version of Ada, make sure Ada wasn't in the middle of something, and then deploy the update. Once Ada was up and running again, Claude would @Ada-sen, explain the test it wanted to run, and wait for results.

Once a bit of functionality works, Claude checks in with Ada to see what could make it simpler and easier to use. Ada typically kicks off some subagents to see whether they stumble through the new feature and then reports back.

At one point, I wrote to Claude "I've got to get to bed. When this project is done, check with Ada to see whatever quality of life features you could build." I woke up to a laundry list of about a dozen harness improvements. Overnight, Claude had solicited a wishlist from Ada. Ada ranked a bunch of requests. Claude ordered them and wrote out a spec for everything it was comfortable building. Ada reviewed the specs, then Claude built the features. Ada tested them out and requested changes. Claude updated the features until Ada was happy. Once Ada signed off, Claude merged the changes to main and wrote up an after-action report for me.

It's been pretty magical to watch.

2.1.263

CHANGELOG · 06 SEP 2026 · SOURCE

- Bug fixes and reliability improvements

The case that's a little more complicated is when the agent wants to do something that requires a browser, like logging into Gmail as themselves or using a service that has no API. For a bunch of reasons, the MITM rewrite trick does not work as reliably in a browser context. Often the site/app's javascript plays games with the password to hash the password before sending it. Or does some other weird validation. Or is actually sending it to a different backend service. So far, the best I've come up with keeps the credential out of the agent's transcript and prevents unintentional disclosure. In these cases, the subagent running in the container generates a random temporary password, uses superpowers-chrome to fill it into the password field, and then runs a command that asks the arbiter agent (running in its own container) for help. The arbiter, if it decides the request is reasonable, instructs a helper tool to log into the subagent's container and replace the random password with the real one. It's not perfect and there's more we can do to lock things down, but it's a better first pass than anything I've had to date.

What actually got me writing today was not this architecture, but the *development pattern* I've been using to build this. On the one side, I've got a Claude Code session. I hooked it up to our Slack instance with a self-provisioned commandline slack client Drew built called Slackline - Slackline is designed to make it easy for agents that don't *live* on Slack to participate in discussions there. It has primitives for sending messages, reading chats, waiting for replies, etc. The whole command-line slackline tool has been built for agents as its primary user. Sure, a human *could* use it to use slack from the commandline. And sure, it has enough of a json api that you could use it as part of traditional automation. But it's *for* agents.

It took a bit of iteration to get to the point where the new Sen 2.0 agentic colleague harness was live on Slack. But once it was there, I could talk through its experience of its tools and memory and credentials and workspaces. And the issues showed up pretty quickly.

The "right" kind of context compaction for a coding agent is desperately wrong for a long-lived persona that might be engaging in multiple simultaneous conversations. And the only way to see if the credentials management tools work is to try to do something crazy, like sign into GitHub from a subagent session.

And sure, I could take this feedback into a Claude Code (or Codex) session and work it through. But I've got too many projects in flight and there's very little reason for me to be in the loop on most of this work. I would just slow it down. So I told Claude Code to use slackline to talk to "@Ada-sen" about the work it was doing.

Claude pinged Ada in #bot-testing to describe the work we'd just done and to ask Ada if it could test it out. Ada dutifully fired up a subagent and tried to log into GitHub. Claude watched the logs to see if the credential proxy did the right thing. It did not.

How do you know which is which? If someone is clearly ferociously competent, they're always going to have *some* power, since upper management tend not to ignore useful tools. But you can't rely on competence as your only guide. Some managers are powerful for other reasons: they're friends with the CEO, or they have strong relationships with other groups like legal or sales, or they're simply willing to do whatever upper management wants done.

One signal is who's leading the important projects. Read your CEO or CTO's internal updates and pay attention to the projects that are called out by name. Organizations tend to give key tasks to trusted lieutenants. If a manager is leading an area that's never under the spotlight, they probably don't have enough clout.

Another signal is hiring. Is a manager's team growing or shrinking? Particularly post-ZIRP, headcount is a rare and precious resource. A manager who's able to get it is likely a powerful manager, or at least is reporting to a powerful director or VP (which often amounts to the same thing).

At all costs, avoid making powerful enemies

First, you should try not to make any enemies at all. Most software engineers who get "playing politics" wrong do it by needlessly alienating people: by being rude, unhelpful, abrasive, making non-technical people feel stupid, and so on. This post isn't really about that. I'm assuming that you can figure out how to be a generically pleasant person on your own.

However, **competent software engineers will make some enemies**. If you're out there making projects happen, some people aren't going to like the way you do it, and won't be a fan of any compromise you offer. I wrote about this in *Big tech engineers need big egos*: the only way to avoid making enemies is to change nothing, but that's incompatible with doing the job.

Given that, be selective about *which* enemies you make. If you're making a technical decision that's either going to require work from team A or team B, and neither team wants to do it, you should try to pick the team with the least political cover. If you need a powerful VP's team to do something they won't like, try to be maximally respectful about it: get that team's core engineers on-side if you can, or book a meeting with the powerful manager and explain the situation, or (better yet) ask the powerful manager sponsoring your project to go and talk to the other VP for you. (If you don't have a powerful manager like this, consider abandoning your project).

Give way to powerful managers when at all possible. Every so often you really do have to stand your ground — if the system will truly collapse otherwise, or a major customer will have an incident, or if the technical decision really is entirely bone-headed — but almost all cases are not like this. The best advice I've ever gotten about playing politics came from a manager I worked with long ago[2]:

> This is not the hill you want to die on.

When I'm about to pick a fight or say something argumentative, and I'm not 100% convinced it's necessary, I ask myself: is this the hill I want to die on? And it never is.

The three rules about disagreeing with powerful people are:

- Make sure you do it in private
- Be polite
- When they overrule you, stop arguing immediately

Disagreeing in private rarely hurts, if you follow these rules. In fact, it can help. If you can manage to disagree with a manager, get overruled, and then follow their plan without complaining, that can be the best way to gain a powerful friend. But if they think you're going to keep griping about it, or worse still, complain to the rest of the team and foment some kind of rebellion, there's no quicker way to make a powerful enemy.

If you have powerful enemies at a company (for instance, the CTO or an influential VP doesn't like you), **quit**. It's really that bad. I have never seen this situation turn itself around, except in the very rare case where the CTO or VP is already looking for greener pastures and jumps ship. You cannot recover the situation: they have no incentive to give you the chance to change their mind, and they have almost unlimited ability to screw you on promotions, raises and layoffs.

That's why this piece of advice is second in the list. If you aren't helpful or if your contributions are invisible, you can work on that and fix it. But if you've made powerful enemies, you're done for.

Help powerful people as best you can

Just as it's fatal to make powerful enemies, it's very useful to make powerful friends. How can you do this? Remember you're a palace guard, not a great lord: you make friends **by doing your job**. However, you can choose to do your job a little more proactively and diligently when you're doing it for someone with political clout.

folder it shares with me. When we moved Sen to the cloud, I lost one of my favorite dumb features - Sen used to be able to print things directly. Sen (v1) was built on top of the Claude Agents SDK.

For the past month or so, I've been doing initial bringup of a "new" iteration of Sen, intended to be more of a colleague than an assistant. This time around, I'm building it from the ground up on top of an agents SDK I control. Lace got its start in May 2025 as my take on a command line coding agent. Over the past year, it's mutated a few times, first growing a web interface, then flipping to an ACP-derived wire protocol so you can connect any client you want to it. It speaks a bunch of different flavors of model provider API, manages caching, subagents, tools, skills, and all the other stuff you might expect. It has supported subagents in isolated containers for the better part of a year. One of the latest things to land has been the ability to run subagents locally and to project *all* of their tools into a container, such that the agent believes it is running in that container.

The thing I've been working on over the past few weeks is the credentials story for the new Sen agent running on top of Lace. Right now, I'm building with a design that assumes the agent has its own credentials for any service that it accesses, not yours. (You wouldn't share your email account or GitHub credentials with a colleague, right?) As of now, nobody has solved Simon's Lethal Trifecta - If a single agent has access to private information, can communicate externally, and is exposed to untrusted content, there is no structural way to guarantee that that agent can't be suborned. So the name of the game is compartmentalization and risk *reduction*.

The architecture I've landed on for now:

- The main agent does not have the ability to directly communicate externally.
- The main agent is able to communicate with ephemeral subagents that *do* have the ability to communicate externally
- No agent has intentional direct exposure to credentials (with one very real gap that I haven't solved yet)

All credentials for the agent live in a 1Password vault. All outgoing https traffic goes through a onecli-style transparent MITM (man in the middle) proxy. When a subagent wants to do something that requires a credential, it runs a command that asks an *arbiter* agent running in another container for permission to use that credential. If the arbiter decides the request is reasonable, it provides the subagent with a random string they can use in place of the requested credential. When the agent uses that random string in an outbound request to the right remote host, the MITM proxy rewrites the random string into the real credential on the fly.

At Prime Radiant, we've got a bunch of agents in our Slack. I want to tell you a bit about how we've started shipping some changes to internal production with an "agentic user in the loop." Letting an agent work directly with the implementer building its runloop and tooling is getting us higher-quality tools and experiences for the agent without a human in the middle of a game of telephone.

But first, a bit of backstory on how we're using non-coding agents at Prime Radiant.

One of them, Scribble, is just there to listen for when someone reports a problem or provides a new bit of information. It opens tickets for the issues and updates an internal wiki with the information. (It also logs our daily standups and keeps us honest about whether we finished what we said we were going to do the previous day.)

Another, Nora, is our junior go-to-market "person." Nora is built on top of Nanoclaw and is still learning her craft, but is surprisingly helpful, partially because she considers herself a team member, not just an assistant. I've put a bunch of time into tuning her prompting and constitution. She gets a ton of value out of knowing that she journals obsessively. (I should note that I don't tend to gender my agents, but Nora picked a name and a gender and was very clear about that and...who am I to argue?)

I think the most surprising interaction I've had with Nora was the morning I woke up to a slack DM telling me that I needed to be speaking at more conferences. (I do.) She pointed out that the CFP for a particular AI conference had closed a week earlier, but that the program chair was known to take late submissions of interesting talks by DM. She'd taken the liberty of drafting a DM for me to send. I didn't actually *send* the DM, but I should have.

We have another agent, "Spec-together" that was a first prototype of what has become our new multi-player Brainstorm app.

And we have a small fleet of "Sen" personal assistant agents. They're **claw-esque*, but date from before openclaw was a thing. Each one has been trained to be an executive assistant, using a set of skills built on some of the resources that excellent human EAs use to learn their craft. Sen triages my mail, puts together a daily brief, does research tasks (both for me and independently), interacts with Linear, etc.

They have the standard set of affordances - They can chat on Slack. They use tools. They've got a task scheduler that can wake them up and re-inject a prompt they set for themselves at a specific time or on a schedule they define. They can use and author skills. Mine has a dropbox

One obvious application of this principle is that **you should answer Slack messages from powerful people immediately**. If you see an ordinary Slack question pop up while you're doing some task, it's okay to get to it when you get to it. In fact, it's ideal *not* to respond to all questions immediately, so you don't set unreasonable expectations (and so you don't seem like you're sitting around doing nothing). But when a VP comes in with a question, don't make them wait: answer the question immediately. If the question requires research, send a "let me look into that right now" message, then do the research. This is the easiest way to get a reputation for being helpful[3].

Another way to do this is to **lean in on important projects**. Suppose you do ten projects in a year. Eight of them are normal, low-priority projects, and two of them are high-profile (say, finishing some big feature before your company's yearly conference). It's a mistake to allocate your effort equally to all ten. I wrote about this at length in *Doing nothing at work*: you should be operating at 80% capacity (or less), so you can then ramp up to 120% when it really matters.

Pay attention to the narrative that powerful people are trying to push. Here are some potential narratives:

- We've had a lot of turnover and reorgs lately, but we're all starting to pull together as a team now
- Isn't it great how focused we all are on reliability work after last month's incident?
- The conference this week is the most important thing, so we're all being very careful not to break anything
- We're an AI-forward team that's looking for the best ways we can leverage LLMs into our team processes
- Although this project had a rocky start, we're now all aligned on the way forward

You don't necessarily have to jump in and start cheerleading, but you should at least not do anything that you know is going to make the narrative look weak. For example, on that last point, it's foolish to openly argue that the project really was fine all along. Bring it up privately, not publicly, or you risk ruining some clever piece of propaganda that the manager in question is trying to push on the rest of the organization[4].

Finally, an underrated way to help powerful people is to offer them social support and information. Slack messages and planning emails might seem unimportant to you, but powerful people often live in that environment: their primary tool is writing messages like these, just like your primary tool is writing code. Reading and responding (in a supportive way) to these messages is something that most engineers don't bother to do, but it goes a long way.

Likewise, dropping a senior manager a line now and then (say, a heads-up that a particular project landed successfully, or that you got good metrics about some feature) is surprisingly helpful. Senior managers live in an information-poor environment: for them to learn something about a team's work, that information has to bubble up through several layers of interpretation and summary. In my experience, they're appreciative of being drip-fed the occasional piece of information, so long as you keep it brief and relatively rare.

Make sure they know you're helping them

If you're directly responding to a VP's Slack messages or DMing them information, they know you're the one doing it. But if you're just doing your job and working hard on projects they care about, they might not notice. **Being invisible is probably the most common way engineers fail at playing politics.**

Fortunately the fix is simple: tell people what you're doing. If you fix an important bug for a launch, write a message in that launch's Slack channel saying "hey, I just fixed this bug". What if you don't like bragging? Get over it. You have to be comfortable publicly telling people what you've done. You should also keep a brag document so you can repeat all of this at review time.

Another, subtler way to do this is to gain the trust and respect of the powerful engineers in your area. Senior managers will always have a few trusted engineers they rely on to assess technical questions. They will ask those engineers what they think about you, and will broadly trust those answers. The good news is that if you're competent and useful, those engineers will already value you, so you don't have to do anything special: just be good at your job.

Technical power

Is playing politics all about sucking up to senior managers? Basically, yeah. A less cynical way to describe it would be "aligning with the values of the company". If you think your company is doing good things, you should want to do that anyway! In any case, what that comes down to is figuring out what the people in charge want, giving it to them, and making sure they see you doing it. However, there's still some scope to get what *you* want out of the deal.

"How does this apply juniors? You can't expect them to really do all of that?"

I've been asked these questions, or variations of them, after I shared the thoughts above and here's my answer.

I do *not* expect juniors to *do* all of these things right away. But I would expect them to read through the list and *aspire* to one day be able to do all of these things. Until then, they can and should ask for help.

In fact, I don't expect *anybody* to *always* do *all of these things* for *everything*. It's a mental checklist of things to consider — problem, edge cases, tradeoffs, deployment, customers, messaging, … — but for quite a few things there aren't edge cases to consider. Or big tradeoffs to weigh. Or deployment is a solved problem. And maybe someone else does the messaging for you.

And I imagine that most of these things you shouldn't even consider when you work in a company with, say, 5000 employees. I've never worked in a company that large, only startups, so I can't speak to how to successfully ship a software feature at Apple, end to end.

But when you work in a small company in which there's only a single department, when you want to build things you're proud of, when you work with me and you say own something, I expect you to keep these things in mind and run through them before you declare something as done.

You know what *you* should own? A subscription to this newsletter:

Some new agentic patterns

MASSIVELY PARALLEL PROCRASTINATION · 08 JUL 2026 · SOURCE

You can also read this post on Prime Radiant's blog.

Almost all of this post was written on June 1, before Anthropic shipped Claude Tag. Nothing I'm writing about uses (or is based on) Claude Tag

- How would we announce this? How do we communicate it? Can you picture it? How does it fit into the larger picture of our roadmap? Questions or concerns in that area — push back! ask!

- Do the work, with precision, with care, with a sense of urgency, with calmness. Do not half-ass things. Before you merge, ask yourself: am I proud of this? would I show this to John Carmack and say "here's what I built, under these constraints, with these tradeoffs?"

- Test it manually. Yes, there's automated tests. But in 99% of cases you can manually test or confirm that what you built works: you can run it yourself, you can ask an agent to run through test scenarios, you can poke at the data before and after, you can take screenshots, you can make a demo. Are you sure that what you did actually solves the problem?

- Make sure it lands in production and *works in production*. Is it deployed? Did the deploy fail? Do you need to activate a feature flag? Does the feature flag work? Can you use it in production? Can you confirm it's actually deployed?

- If you think your colleagues needs to know about this change, because it's new feature they should all test, or it's a new convention in codebase, or maybe it's a tricky thing everybody needs to be aware of, or something else: let them know! Do not underestimate peripheral vision: knowing that person X yesterday changed the behavior of how Z works might save person Y three hours of debugging today when a bug report related to Z comes in.

- Do customers need to know? Who reported the bug? Who's blocked? Let them know.

- Does the world need to know? Announce that it's out.

- Are there follow-ups? Do you need to check on what you shipped in the logs? A week later maybe?

Yes, that's a lot. And there's actually more, because I'm sure I forgot some stuff.

But that's how you build a product in a small team. We don't have PMs, we don't have a Q&A department. We're small, but we're *great*, we can do all of that.

And it's always okay to ask for help, it's okay to ask questions, it's okay to redo things and triple-check. What's not okay is to implicitly assume that someone else will do the things here that you haven't thought about.

I said earlier that software engineers do not wield organizational power. However, that doesn't mean you're powerless. Technical ability is a source of real power, if a delicate and unreliable one. The movers and shakers in tech companies are utterly dependent on technical people to implement their vision and to give them clear answers about the system.

There are many subtle ways you can leverage this. One I wrote about in *How I influence tech company politics as a staff software engineer* is to wait until important people at the company want to do something (say, improve reliability), then offer them a technical plan that does it your way. Another one is to become so useful that you're actively in demand to lead projects, and then run the project how you want.

You probably won't be able to change the company's grand strategy. But how that strategy is *implemented* has a lot of specific technical detail, and you can put yourself in a position to decide on those details.

Conclusion

Playing politics isn't about plotting and scheming, and it isn't just about being a friendly, likeable person (although that helps). It's about figuring out how your company actually operates: who makes the decisions, who gets consulted, what behavior gets rewarded, and so on. The most basic way to do that is to **figure out who is powerful, get out of their way, and (if you can) help them get what they want**.

1. Obviously the exact titles depend on your company. One person I'm deliberately leaving out is your own manager. In general don't think your relationship with your own manager counts as "playing politics": that's just you getting along with another human being. An exception to that is if you report directly to a powerful director or VP.

   ↩

2. Ironically, this manager struggled to take his own advice.

   ↩

3. Note that you actually have to be able to answer their question accurately in order to do this. If you're not competent enough to be useful to powerful people, you will struggle to befriend them.

   ↩

4. For instance, maybe the CEO is convinced that the project was in bad shape because of something he heard, and the manager in question knows it's easier to sell "yes, but we turned it around" than "no, you misunderstood, everything was always fine". If you complicate that process, you risk the CEO thinking that the project is still bad and cancelling it.

   ↩

The lies I used to tell myself

CATE HALL · 23 JAN 2026 · SOURCE

1. What doesn't kill you makes you stronger.

When I was 31, my husband of four years, who I loved more than my own life, left me completely out of the blue. We'd had an incredible, epic romance — drawn together like magnets, we married on our third date and spent the years that followed burrowing ever closer together. Maybe that was the problem — I can't say for sure, because I never got a satisfying explanation out of him. One day I was the protagonist of a Princess Bride-worthy love story; 48 hours later, he moved out.

From the time we met, I had never contemplated the possibility of living without him, and I didn't want to. I wanted to die for a long time — at least a couple of years. But as I slowly learned to sequester away thoughts of him and the emotions they brought up, I became convinced of a silver lining: The experience would make me impenetrable. If I survived it, I would know I could survive anything, and nothing would be able to hurt me again.

I shared it before, but not here, because I didn't think too much of it. Then today, someone said their CTO shared my post with them and I thought: well, now I have to put it in the newsletter, don't I? So here we are.

Below, after the Slack post, I added some thoughts on juniors on how I see this advice applying to them.

Just had a (great) conversation about ownership and engineering here and I realized that I often use the phrase "ownership" or allude to it, but haven't explained what "ownership" means to me in a while.

So, ownership.

If I ask you "can you own this?" or "can you take care of this?" or "are you on it?" — what I'm doing is I'm asking you to *own* it, to own the solution of a problem from end to end. From "we have a problem" to "we don't have to think about it again."

That means, when you say that you're owning something, the expectation is that you…

- Think about what the problem actually is. Maybe you already have a solution in mind, without having thought about what we're actually trying to solve here. Maybe you think "the problem is that we need to migrate from using X to using Y", but that's not a problem, that's a solution. The problem is likely something like "performance is bad", "it's not stable", "it fails for customer x". Maybe there's other possible solutions to that? Think about those. What are the tradeoffs? What's the best solution to go with considering these tradeoffs?

- Think about edge cases. What are they? Which ones are important? Which ones can we ignore?

- Think about failures. Network failures are a given, for example. How do we handle them? Retry? Well, how often? How long?

- Think about data flow. How much data is involved here? Does data need to be migrated? Cleaned up? How can I get my hands on data to properly test this? What invariants are in the data? What assumptions do I have about the shape of the data that I haven't confirmed yet?

- Think about how you'd test this. How can I know that what I built is correct or not? Are tests enough? Do I need to manually poke at things? Is the difference visible on a screenshot or in a video?

rising income, no plateau in sight. The myth persists because it serves a psychological function: It helps people without money feel better about their situation, and helps the wealthy downplay their advantages. But it's not true.

8. Intuition is fake and rationality solves everything.

I used to be horribly judgmental of people who made decisions based on intuition, as in, "I just do what feels right." And then I noticed that whenever I overrode my intuitions, I made *terrible* decisions. I hired the wrong people, started projects with the wrong people, dated the wrong people, ate food that made me feel bad, did forms of exercise I found unfun and injurious. I don't have a satisfying theory for why this is, but I know what happens: When I shift from emotional intuition to a logical story about a decision, it's much easier for my reasoning to get hijacked by plausible-sounding directives that turn out to be nonsense.

It took me a long time to learn this, and that might have something to do with my training in law and poker. Overriding intuition in favor of explicit reasoning makes sense in contexts that reward systematic rationality, where you will get separated from your money if you count on gut feelings. But most of life isn't a closed system with enumerated rules and clear outcomes. If you try to explicitly name all the factors that make someone a good friend or co-founder, you might come up with some good indicators, but your crude map will also mislead you about the territory.

This is why "wisdom is knowledge that can't be transmitted." Wise people have navigational skill, not a long list of rules.

Sign up to be notified when my book, You Can Just Do Things, is available for purchase.

Ownership

THORSTEN BALL · 08 JUL 2026 · SOURCE

Thoughts on ownership I sent to the Amp team in internal note

The following is an internal Slack message I sent to my Amp teammates after I had a conversation with one of them about ownership. It's only lightly edited.

It took the better part of a decade and another deeply identity-shifting relationship, with the man who became my second husband, for me to see that I had it all wrong: Impenetrability means avoiding the parts of life that are most life-like. There's a reason the visual metaphor for love is an arrow.

The near-fatal catastrophes of our lives don't always make us stronger. Surviving one hard thing does provide evidence that you can survive other hard things, which is easy to *mistake* for getting stronger. But in reality they often just make us brittle, make us dissociate and flee from the situations that remind us we can be broken.

2. When you know, you know.

Mario Gabriele of the Generalist had a great "50 Things" style post a few months ago that I found myself nodding along to almost the whole way through. The glaring exception was Number 33: "Love is easy. The difference between bad relationships and meeting your spouse is not a matter of degree, but category. Compatibility is unignorable and effortless."

I believed this completely, once. I've since come to believe that what's unignorable and effortless is chemistry. Chemistry can be powerful enough to turn your life sideways, but it's not the same thing as compatibility. Compatibility is litigated on the timescale of years and decades; it's not a momentary feeling but a description of what happens during an extended course of change. Do you grow toward one another, or do you grow apart? Does the relationship help you move closer to the person you aspire to be?

I "just knew" with my first husband. I wasn't sure with my second. Four years in, the signs have flipped.

3. If it were a good idea, someone would be doing it already.

There's a belief, especially common among wonky types, that you don't find $20 bills on the sidewalk — if an opportunity were real, someone would have seized it already. This sounds worldly and wise, but it's actually a form of defensive thinking. It assumes the current way is best, that everything worth doing is being done.

In my experience, this is just false. When I started playing poker, I discovered that physical tells were basically a wide-open field — despite the fact that most pros believed the topic was played out. If you think you see an opportunity others have missed, you should have a story for why they're missing it, and you should ask around to stress-test that story. But if no one can give you a satisfying reason, don't assume one exists. Sometimes the answer really is just inertia or fear.

4. If it's worth doing, it's worth doing well.

Almost nothing is worth doing "well." Many things can and should be done to a minimal standard of quality, because anything more is a waste of effort that can be better allocated. The exceptions aren't things that should be done "well," but things that should be done to an exceptionally high standard, probably a much higher one than you see modeled by the people around you.

This is not semantics; most people really "do everything the way they do anything" — whether that's in a low-effort or high-effort way. But the point of phoning it in on a lot of stuff is to save your energy to expend on the 1 in 1000 things that really matter … and then to actually spend it.

A recent example: I decided that I want to narrate my own audiobook for *You Can Just Do Things*. When I told a friend in the industry this and asked whether they knew any good coaches to work with, they said that hiring a coach wasn't normal or necessary, because my publisher would decide based on a sample whether I was good enough at reading to do it, and if so give me a producer to work with day-of. But it "1 in 1000" matters to me whether the audiobook is so good no one would ever think of returning it on account of the narration, so I'm not interested in the normal-shaped solution that constitutes doing it "well," I'm interested in the one that causes me to come to mind when someone asks this question. In this case, that means hiring multiple coaches and practicing for months.

5. If you like your job, you're doing something wrong.

Back when I was more alienated from my emotions, I thought about work like this: You have a budget of energy to spend, and the goal is to efficiently convert that energy into impact. You identify the activity that generates the highest impact per unit of energy, and then you spam that button for as many waking hours as you can stomach. Is it any wonder I was always burning out?

What I failed to understand is that energy is not fixed. It's dramatically affected by what you choose to do — some activities increase your energy over time, others deplete it. This feels so obvious to me now that I feel kind of stupid including it here, but I know many people (they are endemic to the Bay Area) who still labor under the false belief. Whether you enjoy what you're doing affects your ability to do it year in and year out, and dismissing this as "selfish" doesn't make it untrue. As my husband puts it, the best use of effortful motivation is to engineer your life to require less of it.

6. "All people are insane. They will do anything at any time, and God help anybody who looks for reasons."

This line, from Kurt Vonnegut, used to neatly encapsulate the way I understood other people — which is to say, not at all. Trying to figure out why people did what they did, or imagine what they'd do next, seemed like a fool's errand. I was judgmental about it, too: I assumed this was a problem, not with me, but with everyone else.

I didn't fundamentally move beyond this belief until recent years, when I came into contact with the Enneagram. I imagine there are other personality typing systems that can do something similar, but the Enneagram was magical for me because it clued me into the fact that people can have very different motivational systems from my own that are still internally coherent and produce good things in the world.

People's behavior was previously incomprehensible to me because I was imputing my own motivational system to them, trying to figure out what would cause me to act the way they did — essentially, concluding that *their* actions made no sense in light of *my* goals. But people have wildly diverse emotional hardware, which determines what's rational for them. And my particular personality — independent, moralistic, systematic, challenge-seeking, competitive — is unusual, so I was especially poorly placed to measure others by the yardstick of myself.

This shift in perspective was dramatically good for me and for my relationships with other people. It let me see the ecosystem of psychologies as a kind of miracle — wow, it really does take all kinds — rather than a prompt for cosmic horror. And it gave me confidence that, if I invested in understanding people more deeply, they would eventually cease to be total mysteries, prone to bizarre and unreasonable behavior. As far as I can tell, this is the better part of emotional intelligence.

7. Money can't buy happiness.

The good version of this advice is: You'll still have to work at being happy, even if you're rich. Money in your bank account won't do it for you, and wealth comes with its own pathologies, like endless status-chasing. But the common gloss — that money stops mattering past some modest threshold — is basically false.

You may have heard there's research showing money doesn't buy happiness. That research was explosively popular precisely because it was counterintuitive — it betrayed the commonsense intuition that more money means more freedom, more options, more ability to help people you care about. Turns out it was also riddled with methodological problems. More recent work by Matthew Killingsworth (nominative determinism FTW), using large-scale studies that pinged participants about their happiness at random moments, found steady returns to well-being with